PROJETO PEX · MANAUS/AM · 2026

Smart Parking

Monitoramento inteligente de vagas de shopping

Menos tempo procurando vaga. Mais tempo no shopping. App do motorista, painel do gestor
e sensores IoT — validados de ponta a ponta, com falhas zero na véspera da entrega.

Breno Santos

Autor · Liderança · Desenvolvimento

Gabriel Almeida

Desenvolvimento

Jonas Almeida

Desenvolvimento de UI

FastAPI · React Native (Expo) · PostgreSQL · Docker · MIT License



O PROBLEMA

Estacionar é o pior pedaço de ir ao shopping

~20 min

tempo médio girando até achar vaga em shoppings cheios

0 dados

gestor decide lotação no olhômetro; histórico não vira relatório

RISCO

vagas PCD/idoso ocupadas indevidamente — direito desrespeitado

fila no portão

sem reserva nem leitura de placa, pico vira gargalo

Shoppings de Manaus operam com planilha e catraca manual. O motorista paga em tempo, combustível e estresse; o shopping, em ocupação mal distribuída e receita não medida.

A SOLUÇÃO

Um sistema, três pontas: motorista · gestor · sensor

App do motorista

Mapa de vagas por andar em tempo real, "próxima vaga" atribuída
por algoritmo, reserva, cancelamento, perfil e pagamento.

Painel do gestor

Ocupação, CRUD de estacionamentos/andares/vagas em lote, relatórios
e faturamento, usuários, pagamentos e simulador QR de portaria.

Sensores IoT (RF-45)

Dispositivos reportam livre/ocupada sem login humano, com chave única,
idempotência por sequência e revogação instantânea.

Diferencial de cidadania (RF-47): o algoritmo nunca entrega vaga PCD a quem não se declarou,
bloqueia reserva indevida com explicação e, sem PCD livre, rebaixa com aviso transparente — sem lockout.

O PRODUTO · MOTORISTA

Quatro toques entre "cheguei" e "estacionei"

Explorar

shoppings, distância e vagas livres ao vivo

Mapa

andares e células por status e tipo (PCD/idoso/moto/elétrica)

Próxima vaga

atribuição automática — nunca PCD às cegas

Reserva

confirmada; cancelar devolve a vaga ao mapa



Perfil com placa, login por e-mail OU telefone e cache offline no subsolo. Seed demo: 8 shoppings de Manaus e 420 vagas em 4 andares só no Amazonas Shopping.

O PRODUTO · GESTOR

Operação e dinheiro no mesmo painel

Operação

criar/editar estacionamento, andares, vagas em lote com códigos sequenciais e status manual quando o sensor falha — tudo reflete no app em segundos.

Financeiro

pagamentos com registro, geração de Pix copia-e-cola e confirmação; relatórios de ocupação e faturamento por período e estacionamento.

Controle & sensores

usuários por papel (admin/gestor/financeiro), emissão e revogação de chave de sensor e simulador de QR para a portaria.



Multi-tenant desde o MVP: o gestor vê só o próprio estacionamento — escopo aplicado no backend, não no olhar.

COMO FOI CONSTRUÍDO

Arquitetura enxuta, camadas limpas

Frontend	React Native + Expo SDK 57 · TypeScript · NativeWind — um código para Android, iOS e web (11 telas)
Borda	nginx (dev/prod): proxy, cabeçalhos de segurança, CORS restrito · docker compose up
API · 71 rotas	FastAPI + SQLAlchemy · Pydantic · 4 routers (motorista, admin, auth, sensores) · gunicorn
Dados	PostgreSQL 15 (SQLite no dev) · migrações Alembic · seed com 8 shoppings
Dispositivos	ingestão HTTP com X-Device-Key (SHA-256 no banco) e idempotência por seq — sem JWT

19 pull requests squash-merged · commits rastreáveis por marca de entrega · CI obrigatório verde.

SEGURANÇA & CONTROLE DE ACESSO

Não é promessa de design — é matriz executada

Identidade real

JWT curto + refresh; revogação por jti e por "usuário desde T" —
trocar senha ou bloquear derruba todas as sessões.

Anti-replay

assinatura HMAC obrigatória em toda mutação + janela de timestamp:
Postman/curl repetindo requisição alheia é barrado no middleware.

Defesa em profundidade

escopo por estacionamento no servidor, cabeçalhos de segurança,
TLS documentado (teste Wireshark), validadores de payload.

401 sem token · chave revogada · sessão morta

403 papel errado não opera vaga nem reserva PCD

410 QR revogado não reanima

429 brute-force e flood com Retry-After

Cada código acima tem checkpoint de E2E que o produziu ao vivo (matriz 27/27 · jornadas D3e, D5a, D6e, D7d).

QUALIDADE · COMO PROVAMOS

Falhas zero não é adjetivo. É exit 0.

```
$ ./scripts/e2e_validacao.sh --full

✔ A · conexão (porta, LAN, CORS, .env)

✔ B · smoke MVP ..... 10 OK / 0 FAIL

✔ C · matriz de acesso .... 27 OK / 0 FAIL

✔ D · jornadas D1–D7 ..... 28/28 ✔

✔ E · 51 pytest + ruff limpo

✔ F · tsc limpo   → exit 0
```



E2E de verdade

~90 asserções HTTP reais contra a API no ar; cada estado é lido de volta por GET (cancelou? está livre no mapa).

Caça-bugs

o E2E achou e matou os 3 últimos defeitos: sessão-fantasma, Alert no-op na web e flash branco dos botões.

ROADMAP · AS 5 MARCAS DO PEX

Entregas em ritmo, com uma adiantada

10/09	**1 · Banco de dados modelado** Modelagem completa: usuários e papéis, estacionamentos → andares → vagas, reservas, pagamentos, sensores, revogação.	✔ entregue
22/10	**2 · MVP operacional completo** API 71 rotas + 11 telas + painel; RF-47 de ponta a ponta; IoT; QR; Pix; nginx+Docker; E2E exit 0.	✔ esta entrega
02/11	**3 · Relatórios prontos** 7 endpoints de ocupação e faturamento consumidos pelo painel com escopo por estacionamento.	✔ adiantado
09/11	**4 · Testes & documentação** 51 testes + E2E versionado + docs de integração/arquitetura/segurança; roteiro de banca em curso.	em curso
12/11	**5 · Entrega Integrada Funcional** Apresentação ao vivo: sensor → mapa → reserva → cancelamento → relatório, no docker + Expo Go.	próxima

COMO USAR · RODA EM QUALQUER MÁQUINA

Três terminais e está no ar

```
$ docker compose -f src/docker-compose.yml up
# API+DB no :80 via nginx, app web servido

$ ./src/run-dev.sh
# sem Docker: venv + migrações + seed + API :8000
```

```
$ cd src/frontend/smart-parking && npx expo start
# celular na LAN: a URL se resolve sozinha

$ ./scripts/check_conexao.sh --fix
# acha e valida o caminho sozinho
```

Motorista

cadastre-se ou use motorista@smartparking.com do seed; explore o Amazonas, peça a próxima vaga e reserve.

Gestor

admin@smartparking.com / admin123: crie um andar, gere 40 vagas em lote, abra o simulador QR.

IoT

emita a chave do sensor no painel e POST /api/sensores/eventos — a célula muda no mapa do celular em tempo real.

O QUE VEM COM O USO

Quanto mais roda, mais inteligente fica

O ciclo de dados (já plantado)

- ✔ cada reserva, cancelamento e evento de sensor engrossa histórico estruturado
- ✔ histórico → ocupação por andar/hora/dia nos relatórios; amanhã, em previsão
- ✔ previsão → "chegue até 14h: 87% de chance de vaga" e preço dinâmico por pico
- ✔ no-show do motorista → punição suave e fila de espera por andar

Melhorias básicas da fase 2

- ◉ push: lembrete de expiração, "seu carro está no P2"
- ◉ cobrança por minuto via leitura de placa + Pix (fluxo já existe)
- ◉ catraca real: o QR de sessão e o contrato de sensores são o encaixe
- ◉ fila de espera com prioridade PCD/idoso e reserva recorrente
- ◉ cache offline → fila de sincronização quando o sinal voltar

Nada disso é retrabalho: os pontos de encaixe (histórico, QR, chaves, relatórios) foram construídos nesta rodada.

Obrigado — alguma pergunta?

FECHAMENTO

Smart Parking não é um app de vaga.
É infraestrutura de tempo — e está rodando.

71	28/28	51	exit 0

banca

Pedido à banca

abrir o app, reservar no mapa, ver o sensor mudar a célula, cancelar e o relatório responder na hora. A segurança se mostra pelos 401/403/410 da matriz executada.

Equipe & licença

Breno Santos · Gabriel Almeida · Jonas Almeida — PEX Manaus/AM · MIT
github.com/beh-S4nt05/smart-parking-pex · behsolutionsandautomations@gmail.com